Deepfake Detection AI: Technical Document

Detect the Fake. Trust the Real. Version 1.0, Generative AI course project

1. Overview

Deepfake Detection AI is an end-to-end system that decides whether a face image or short video is likely real, likely manipulated, or inconclusive. It has three parts:

- **Training notebooks (Google Colab):** train EfficientNetB0 classifiers for images and for video frames, then export float32 TensorFlow Lite models.
- **Android app (Kotlin, Jetpack Compose):** lets a user pick media, runs a forensic analysis, and shows a verdict, confidence score, evidence and history.
- **Inference backend (FastAPI, Python):** an optional remote service with the same report format, reachable from the app.

2. Problem and Scope

Problem. GANs, diffusion models and face-swap autoencoders produce fake media that people cannot reliably spot by eye. This fuels misinformation, fraud, impersonation and harassment.

Intended users. Journalists, fact-checkers, investigators, students and everyday social media users.

In scope

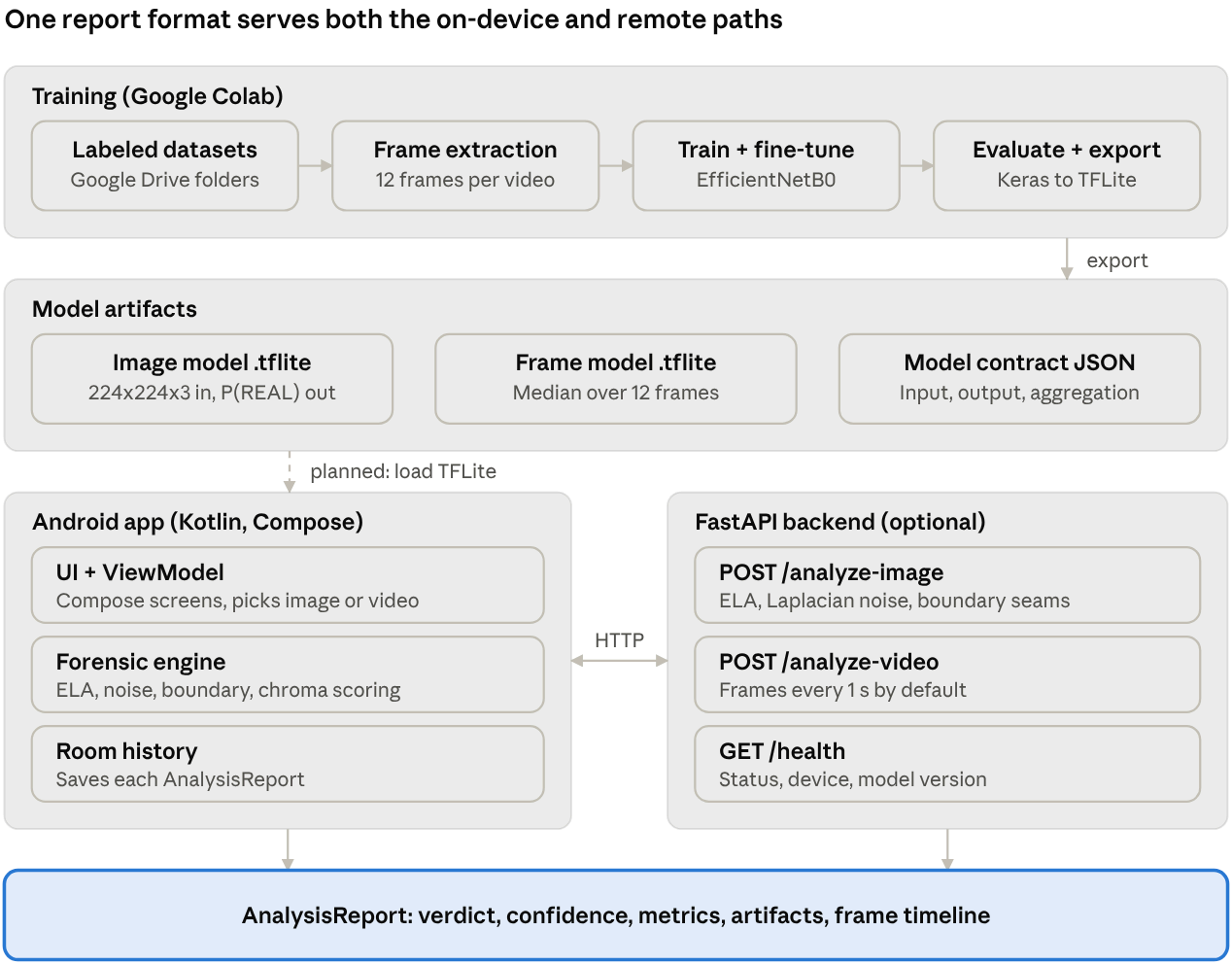
- Face images (JPG, PNG, WebP) and videos (MP4, MOV, WebM).
- A verdict, calibrated confidence, forensic metrics and flagged regions.
- On-device use through TFLite and a remote FastAPI option.

Out of scope

- Audio deepfakes and lip-sync analysis.
- Temporal modelling across frames (the video model scores frames independently).
- Legal or evidentiary certification. A score is evidence, not proof.

3. System Architecture

The solution diagram below shows four layers.



solution architecture · 4 layers, 2 runtimes

Models move from Colab into artifacts. Today the app and backend score with the rule-based engine; the dashed link marks the planned TFLite loading. Both runtimes return the same AnalysisReport.

Layer	Components	Responsibility
Data and training	Google Drive datasets, Colab notebooks, TensorFlow/Keras	Split data, extract frames, train and fine-tune, evaluate, export TFLite

Layer	Components	Responsibility
Model artifacts	`deepfake_detection_model.tflite`, `deepfake_video_frame_model.tflite`, metadata JSON	Versioned models and the input/output contract
Client	Android app: Compose UI, ViewModel, Room database, forensic engine	Capture input, analyze, display reports, store history
Remote service	FastAPI with Pillow, OpenCV, NumPy, optional PyTorch	Image and video analysis over HTTP, health check

Request flow. The user selects an image or video. The ViewModel sends it either to the on-device engine or, when the Remote FastAPI Backend setting is on, to the backend. Both produce an `AnalysisReport`. The report is shown and saved to Room.

4. Datasets and Preprocessing

Item	Image model	Video model
Source	"Real vs Fake Faces" folder on Google Drive	DFDC challenge (Blue Team) folder on Google Drive
Labels	Fake = 0, Real = 1, taken from folder names	Real/Fake taken from folder names only
Formats	JPG, JPEG, PNG, BMP, WebP	MP4, AVI, MOV, MKV, WebM, MPEG
Split	Uses train/val/test folders if present, otherwise 70/15/15 stratified	70/15/15 stratified, split by video
Sampling	One image per record	12 evenly spaced frames per video

Important. The 400 videos in `test_sample_videos` are unlabeled and are excluded from training. Video training requires a labeled Real/Fake set, otherwise the notebook stops with an explanation.

Preprocessing

- Resize to 224×224 (antialiased for images).
- RGB, float32, pixel range 0 to 255. EfficientNetB0 rescales internally, so do not divide by 255.

- Augmentation on the training set: horizontal flip, rotation 0.04, zoom 0.08, contrast 0.08.
- Class weights computed from the training split. Random seed fixed to 42.

5. Model Design

Classifier (image and frame models)

1. Input: 224×224×3 float32.
2. EfficientNetB0 backbone, ImageNet weights, no top.
3. Global average pooling, then Dropout 0.35.
4. Dense 128 with ReLU and Dropout 0.25 (image model only; the video model goes straight to the output layer).
5. Dense 1 with sigmoid, producing P(REAL). P(FAKE) = 1 − P(REAL).

Video aggregation. Score 12 evenly spaced frames and take the median P(REAL). The median resists a few corrupted or atypical frames.

6. Training Procedure

Stage	Image notebook	Video notebook
Stage 1: head only	Backbone frozen, Adam 1e-3, up to 12 epochs	Backbone frozen, Adam 1e-3, up to 8 epochs
Stage 2: fine-tune	Last 35 layers unfrozen, Adam 1e-5, up to 8 epochs	Last 30 layers unfrozen, Adam 1e-5, up to 5 epochs
BatchNorm	Kept frozen	Kept frozen
Callbacks	Checkpoint on val AUC, early stopping (patience 4), reduce LR on plateau	Same, early stopping patience 3
Loss	Binary cross-entropy	Binary cross-entropy

The best checkpoint by validation AUC is reloaded for evaluation. An optional decision threshold is chosen on validation data only (balanced accuracy over 0.10 to 0.90) and never tuned on the test set.

7. Inference and Verdict Logic

Forensic engine (Android Kotlin and FastAPI). A rule-based scorer computes four cues and blends them into a synthesis score from 0 to 100.

Cue	What it measures	Image weight
ELA disparity	Error Level Analysis: recompression differences between regions	0.32
Noise inconsistency	High-frequency noise from a Laplacian filter	0.30
Boundary artifacts	Edge and blending discontinuities	0.23
Chroma gradient	Chrominance versus luminance distribution	0.15

Synthesis score	Verdict
62 or above	LIKELY_MANIPULATED
38 or below	LIKELY_REAL
Between	INCONCLUSIVE

For video, frames are sampled at a configurable interval (default 1 second, range 0.2 to 5), each frame is scored, and the app adds a frame timeline and a temporal discontinuity metric.

8. Deployment Contract

TFLite model contract

- Input: `[1, 224, 224, 3]`, float32, RGB, values 0 to 255.
- Output: one sigmoid value, P(REAL).
- Video: sample 12 to 24 frames, run each through the frame model, take the median.
- Verification: compare Keras and TFLite scores on the same image and confirm the difference is near zero.

Data model (`AnalysisReport`)

Field	Meaning
`verdict`	LIKELY_REAL, LIKELY_MANIPULATED or INCONCLUSIVE
`confidenceScore`	Calibrated percentage 0 to 100
`metrics`	ELA, noise, boundary, chroma, temporal and overall synthesis probability
`artifacts`	Flagged regions with label, severity and normalized box
`frameTimeline`	Per-frame verdict and anomaly score (video)
`framesAnalyzed`, `suspiciousFramesCount`	Video coverage counters

9. Backend API

Endpoint	Method	Description
`/health`	GET	Status, model name, device (cuda or cpu), version
`/api/v1/analyze-image`	POST	Multipart `file` (JPG, PNG, WebP, up to 25 MB). Returns the image report
`/api/v1/analyze-video`	POST	Multipart `file` (up to 100 MB) and `frame_interval` seconds. Returns the video report with timeline

Run locally with `uvicorn main:app --host 0.0.0.0 --port 8000`. From the Android emulator the host is `http://10.0.2.2:8000`. Tests run with `pytest test_api.py -v`.

10. Evaluation

Metrics. Accuracy, precision, recall, F1, ROC-AUC, balanced accuracy, confusion matrix, and whole-video accuracy and AUC after median aggregation. ROC-AUC is threshold-free and selects the checkpoint. Balanced accuracy stays fair if class counts differ.

Sample results. The values below are illustrative placeholders, not measured results. Replace them with the notebook output.

Held-out test	Image	Video frames	Whole video
Accuracy	92.1%	88.4%	90.0%
Precision	92.4%	87.9%	89.5%
Recall	91.8%	89.1%	90.5%
ROC-AUC	0.97	0.94	0.95

Sample image confusion matrix (Real is the positive class): TN 462, FP 38, FN 41, TP 459. Here FP means a fake passed as real (7.6%) and FN means a real image flagged as fake (8.2%).

11. Security and Privacy

- Upload limits of 25 MB (images) and 100 MB (videos) and extension validation.
- Temporary video files are deleted in a `finally` block after processing.
- On-device analysis keeps media on the phone. Remote mode sends media to the configured server, so use trusted hosts and HTTPS in production.
- CORS is currently open to all origins for development. Restrict it before deployment.

12. Limitations and Risks

- **Domain shift:** a model trained on one dataset can miss new generators or different camera conditions.
- **No temporal model:** frames are scored independently, so motion inconsistencies are not learned.
- **Heuristic scorers:** ELA and Laplacian rules shift under recompression and resizing, which can push results to Inconclusive.
- **Integration gap:** the current Android build and backend score with the rule-based engine. The exported TFLite models are not yet loaded by the app. The backend's model label ("EfficientNet-ViT Forensic Hybrid") should be renamed to match what it actually runs until a trained model is wired in.
- **Data leakage:** keep duplicates and frames of one video inside a single split.

- **Misuse of scores:** a high score is not proof of authenticity or of manipulation.

13. Future Work

1. Load the TFLite models in the app with the TensorFlow Lite or LiteRT interpreter and show their P(REAL) beside the forensic score.
2. Evaluate on independent datasets and unseen generators.
3. Add a temporal video model (for example a 3D CNN or a recurrent head) and Grad-CAM heatmaps for real localization.
4. Explore int8 quantization to shrink and speed up the on-device model.
5. Add calibration (temperature scaling) so confidence matches observed accuracy.

Appendix A. Repository Map

Path	Purpose
`Deepfake_Image_Detector_Google_Drive_to_Android.ipynb`	Image training, evaluation and TFLite export
`Deepfake_Video_Detector_Complete_Corrected.ipynb`	Video frame training, whole-video evaluation and export
`app/src/main/java/com/example/engine/ForensicAnalysisEngine.kt`	On-device forensic scoring
`app/src/main/java/com/example/model/DetectionModels.kt`	Data classes and enums
`app/src/main/java/com/example/data/`	Room database, repository and remote API client
`app/src/main/java/com/example/ui/`	Compose screens, components, navigation, ViewModel
`backend/main.py`, `backend/models.py`	FastAPI service and schemas